

DESAFÍO FINAL DE PROGRAMACIÓN WEB I

“Página web interactiva para simular problemas reales del contexto actual”

1. Contexto del desafío

En el contexto actual que atraviesa el país, se observan problemas relacionados con el abastecimiento de carburantes, transporte, alimentos, incremento de precios, rumores de escasez, bloqueos, conflictos sociales y pérdida del poder adquisitivo familiar.

Este desafío propone que los estudiantes de **Programación Web I** desarrollen una **página web interactiva** que permita representar, calcular y visualizar algunos de estos problemas mediante modelos matemáticos sencillos.

La finalidad del proyecto no es tomar una posición política, sino crear una aplicación web educativa que ayude a comprender cómo ciertos factores pueden afectar el abastecimiento, los precios, el consumo y la toma de decisiones familiares o comunitarias.

2. Objetivo general

Diseñar y desarrollar una página web interactiva utilizando **HTML5, CSS y JavaScript**, que permita simular situaciones reales relacionadas con abastecimiento, precios, reservas, transporte o consumo familiar, aplicando buenas prácticas de diseño web y programación básica.

3. Objetivos específicos

El estudiante deberá:

1. Crear una página web con estructura semántica usando HTML5.
2. Aplicar estilos CSS con una paleta de colores adecuada al tema.
3. Diseñar una interfaz responsiva que se adapte a computadora, tablet y celular.
4. Usar JavaScript para realizar cálculos y mostrar resultados dinámicos.
5. Aplicar el DOM para capturar datos, modificar contenido y mostrar resultados en pantalla.

6. Organizar el proyecto en carpetas separadas.
7. Incluir formularios interactivos para ingresar datos.
8. Mostrar resultados mediante tarjetas, tablas, mensajes o gráficos simples.
9. Publicar la página en internet.
10. Subir el código fuente a un repositorio Git.

4. Tema central del proyecto

“Simulador web de abastecimiento, precios y consumo familiar en contexto de crisis”

El proyecto debe representar uno o varios problemas reales mediante una página web interactiva.

5. Tecnologías permitidas

Los estudiantes deben utilizar:

- HTML5.
- CSS3.
- JavaScript.
- DOM.
- Git y GitHub.
- GitHub Pages, Netlify o Vercel para publicar la página.

6. Restricciones del proyecto

No se permite entregar solamente capturas de pantalla o archivos sueltos.

El proyecto debe funcionar como una página web real y publicada.

El estudiante debe entregar:

1. Enlace de la página web publicada.
2. Enlace del repositorio Git.

7. Problemas sugeridos para la página web

El estudiante puede elegir uno o varios de los siguientes escenarios.

Escenario A: Simulador de abastecimiento de carburantes

Situación

Una estación de servicio tiene una cantidad limitada de carburante. La página debe permitir calcular cuántos días durará la reserva según el consumo diario.

Datos que puede ingresar el usuario

- Cantidad inicial de carburante.
- Consumo diario estimado.
- Cantidad de reabastecimiento diario.
- Nivel crítico de reserva.

Cálculos sugeridos

- Reserva restante por día.
- Día en que la reserva llega al nivel crítico.
- Mensaje de alerta si la reserva se agota.
- Comparación entre consumo normal y consumo alto.

Resultado esperado

La página debe mostrar una tabla o tarjeta indicando cuántos días durará la reserva.

Escenario B: Simulador de precios de alimentos

Situación

Los precios de algunos productos de la canasta familiar aumentan durante el mes. La página debe permitir calcular el gasto aproximado de una familia.

Datos que puede ingresar el usuario

- Producto.
- Precio inicial.
- Precio actual.
- Cantidad consumida por semana.
- Número de semanas.

Cálculos sugeridos

- Incremento del precio.
- Porcentaje de aumento.
- Gasto semanal.
- Gasto mensual estimado.

- Diferencia entre antes y ahora.

Resultado esperado

La página debe mostrar cuánto más está gastando una familia debido al aumento de precios.

Escenario C: Simulador de costo de transporte

Situación

El transporte se ve afectado por bloqueos, desvíos o aumento de combustible. La página debe permitir estimar el aumento del costo de traslado.

Datos que puede ingresar el usuario

- Distancia normal.
- Distancia con desvío.
- Costo por kilómetro.
- Número de viajes por semana.

Cálculos sugeridos

- Costo normal.
- Costo con desvío.
- Diferencia de costo.
- Gasto adicional por semana o por mes.

Resultado esperado

La página debe mostrar cuánto aumenta el gasto de transporte cuando existen desvíos o rutas más largas.

Escenario D: Simulador de compras familiares

Situación

Una familia tiene un presupuesto limitado y debe comprar alimentos básicos. La página debe ayudar a calcular si el presupuesto alcanza.

Datos que puede ingresar el usuario

- Presupuesto familiar.
- Lista de productos.
- Precio de cada producto.
- Cantidad a comprar.

Cálculos sugeridos

- Total de la compra.
- Saldo restante.
- Monto faltante, si el presupuesto no alcanza.
- Clasificación del gasto: bajo, medio o alto.

Resultado esperado

La página debe mostrar si el presupuesto alcanza o no alcanza para cubrir la compra.

.

Escenario E: Simulador de rumor de escasez y compras por pánico

Situación

Cuando circulan rumores de escasez, algunas personas compran más de lo habitual. La página debe simular cómo cambia la demanda.

Datos que puede ingresar el usuario

- Demanda normal.
- Porcentaje de aumento por rumor.
- Stock disponible.
- Número de personas o familias.

Cálculos sugeridos

- Nueva demanda.
- Diferencia con la demanda normal.
- Stock restante.
- Mensaje de alerta si la demanda supera el stock.

Resultado esperado

La página debe mostrar cómo un aumento de la demanda puede afectar el abastecimiento disponible.

.

Escenario F: Simulador de pérdida del poder adquisitivo

Situación

Una familia gana el mismo sueldo, pero los precios aumentan. La página debe calcular cuánto poder de compra se pierde.

Datos que puede ingresar el usuario

- Ingreso familiar mensual.
- Gasto mensual anterior.
- Gasto mensual actual.
- Precio anterior y actual de productos básicos.

Cálculos sugeridos

- Aumento del gasto.
- Porcentaje de pérdida del poder adquisitivo.
- Saldo antes y después.
- Nivel de afectación familiar.

Resultado esperado

La página debe mostrar cuánto menos puede comprar una familia con el mismo ingreso.

8. Apartado obligatorio: Casos de estudio para probar

Cada proyecto debe incluir una sección llamada:

“Casos de estudio para probar”

En esta sección, los estudiantes deben presentar ejemplos con datos ya definidos para comprobar que su página funciona correctamente.

Ejemplo de caso de estudio 1: Reserva de carburante

Dato	Valor
Reserva inicial	10000 litros
Consumo diario	1200 litros
Reabastecimiento diario	300 litros
Nivel crítico	2000 litros

Resultado esperado

La página debe calcular aproximadamente en cuántos días la reserva llegará al nivel crítico.

Ejemplo de caso de estudio 2: Aumento del precio de alimentos

Producto	Precio anterior	Precio actual	Cantidad mensual
Arroz	8 Bs	11 Bs	10 unidades
Papa	7 Bs	10 Bs	8 unidades
Aceite	12 Bs	18 Bs	4 unidades

Resultado esperado

La página debe calcular el gasto anterior, el gasto actual y la diferencia.

Ejemplo de caso de estudio 3: Transporte con desvío

Dato	Valor
Distancia normal	10 km
Distancia con desvío	16 km
Costo por km	2 Bs
Viajes por semana	5

Resultado esperado

La página debe calcular el gasto adicional semanal por el desvío.

Ejemplo de caso de estudio 4: Presupuesto familiar

Dato	Valor
Presupuesto disponible	500 Bs
Total de compra	580 Bs

Resultado esperado

La página debe mostrar que el presupuesto no alcanza y calcular cuánto falta.

Ejemplo de caso de estudio 5: Rumor de escasez

Dato	Valor
Demanda normal	100 unidades
Aumento por rumor	40%
Stock disponible	120 unidades

Resultado esperado

La página debe calcular la nueva demanda y mostrar si el stock alcanza o no.

9. Requisitos técnicos del proyecto

La página web debe cumplir con los siguientes requisitos:

Requisito	Descripción
HTML5 semántico	Usar etiquetas como header, nav, main, section, article y footer.
CSS externo	Los estilos deben estar en un archivo CSS separado.
JavaScript externo	El código JavaScript debe estar en un archivo JS separado.
Uso del DOM	Capturar datos de formularios y mostrar resultados dinámicos.
Formularios	Incluir inputs, selects, botones y validaciones básicas.
Diseño responsivo	La página debe adaptarse a diferentes tamaños de pantalla.
Paleta de colores	Usar colores coherentes con el tema del proyecto.
Tipografía legible	Usar tamaños y fuentes fáciles de leer.
Organización en carpetas	Separar archivos HTML, CSS, JS e imágenes.
Buenas prácticas	Código ordenado, comentado y fácil de revisar.
Resultados visibles	Mostrar resultados en pantalla, no solo en consola.
Casos de estudio	Incluir ejemplos de prueba para verificar el funcionamiento.
Publicación web	La página debe estar publicada en

Repositorio Git

internet.

El código debe estar subido a GitHub u otra plataforma Git.

10. Estructura sugerida de carpetas

El proyecto debe estar organizado de la siguiente manera:

```
proyecto-web-crisis/
├── index.html
├── css/
│   └── estilos.css
├── js/
│   └── script.js
├── img/
│   └── imagenes-del-proyecto
└── README.md
```

11. Contenido mínimo de la página

La página debe incluir las siguientes secciones:

Sección	Contenido
Inicio	Título, descripción del problema y objetivo del simulador.
Contexto	Explicación breve de la situación elegida.
Simulador	Formulario para ingresar datos y botón para calcular.
Resultados	Área donde se muestran los resultados generados con JavaScript.
Casos de estudio	Datos de prueba para verificar el funcionamiento.
Conclusiones	Explicación de lo aprendido y utilidad del modelo.
Créditos	Nombre del estudiante o grupo.

12. Requisitos de diseño web

Los estudiantes deben aplicar buenas prácticas de diseño:

1. Usar una paleta de colores adecuada.
2. Mantener contraste entre fondo y texto.
3. Usar botones visibles y fáciles de identificar.
4. Organizar el contenido en tarjetas, secciones o bloques.
5. Evitar saturar la página con demasiado texto.
6. Usar márgenes y espacios adecuados.
7. Mantener una navegación clara.
8. Usar diseño responsivo con media queries.
9. Cuidar la ortografía y redacción.
10. Mantener coherencia visual en toda la página.

13. Requisitos de JavaScript y DOM

La página debe demostrar el uso de JavaScript y DOM mediante:

Requisito	Ejemplo
Capturar valores	Usar <code>document.getElementById()</code> o <code>querySelector()</code> .
Validar datos	Verificar que los campos no estén vacíos.
Realizar cálculos	Sumar, restar, multiplicar, calcular porcentajes o promedios.
Mostrar resultados	Usar <code>innerHTML</code> , <code>textContent</code> o <code>createElement()</code> .
Manejar eventos	Usar <code>onclick</code> o <code>addEventListener()</code> .
Limpiar formularios	Crear un botón para borrar datos.
Mostrar alertas	Indicar si una situación es crítica o normal.
Cambiar estilos dinámicamente	Cambiar colores según el resultado: normal, alerta o crítico.

14. Ejemplo de modelo matemático sencillo

El estudiante puede usar modelos simples como:

Reserva de carburante

Reserva final = reserva inicial + reabastecimiento - consumo

Aumento de precio

Porcentaje de aumento = $((\text{precio actual} - \text{precio anterior}) / \text{precio anterior}) \times 100$

Gasto mensual

Gasto mensual = precio $\times$ cantidad

Transporte con desvío

Costo adicional = (distancia con desvío - distancia normal) $\times$ costo por kilómetro

Stock afectado por rumor

Nueva demanda = demanda normal + demanda normal $\times$ porcentaje de aumento

15. Entregables

Cada estudiante debe entregar únicamente:

Enlace de la página publicada	Debe abrir correctamente y permitir probar el simulador.
Enlace del repositorio Git	Debe contener todos los archivos del proyecto.

Formato sugerido de entrega:

Nombre completo:

Materia: Programación Web I

Título del proyecto:

Enlace de la página web:

Enlace del repositorio Git:

16. Criterios de evaluación

Criterio	Puntaje
Presenta claramente el contexto del problema elegido	5 pts
La página tiene estructura HTML5 semántica	8 pts
Aplica correctamente CSS externo	8 pts
Usa una paleta de colores coherente y legible	5 pts

Presenta diseño responsivo para diferentes pantallas	8 pts
Usa JavaScript para realizar cálculos	10 pts
Aplica correctamente el DOM para capturar y mostrar datos	10 pts
Incluye formularios funcionales y claros	7 pts
Muestra resultados en pantalla de forma comprensible	7 pts
Incluye validaciones básicas de datos	5 pts
Incluye casos de estudio para probar el simulador	8 pts
Organiza el proyecto en carpetas	5 pts
El código está ordenado y comentado	5 pts
Incluye un archivo README.md con descripción del proyecto	4 pts
La página está publicada y el enlace funciona	5 pts
El repositorio Git está completo y accesible	5 pts
Total	105 pts

17. Checklist de autoevaluación del estudiante

Antes de entregar el proyecto, cada estudiante debe revisar lo siguiente:

N.º	Pregunta de autoevaluación	Sí	No	Observación
1	¿Mi página tiene un título claro?			
2	¿Explicué el problema real que estoy simulando?			
3	¿Usé etiquetas HTML5 como header, main, section y footer?			
4	¿Mi archivo CSS está separado del HTML?			
5	¿Mi archivo JavaScript está separado del HTML?			
6	¿Organicé el proyecto en carpetas?			
7	¿Usé una paleta de colores adecuada?			
8	¿El texto se lee bien sobre el fondo?			

9	¿La página se adapta a celular y computadora?			
10	¿Incluí un formulario para ingresar datos?			
11	¿Usé JavaScript para hacer cálculos?			
12	¿Usé el DOM para capturar datos del formulario?			
13	¿Los resultados aparecen en la página?			
14	¿Evité mostrar resultados solamente en consola?			
15	¿Validé que los campos no estén vacíos?			
16	¿Incluí mensajes de alerta o interpretación?			
17	¿Incluí casos de estudio para probar el simulador?			
18	¿Probé el proyecto con diferentes datos?			
19	¿El código está ordenado y comentado?			
20	¿Incluí un archivo README.md?			
21	¿Subí el proyecto a un repositorio Git?			
22	¿Publiqué la página en internet?			
23	¿El enlace de la página abre correctamente?			
24	¿El enlace del repositorio Git está disponible?			
25	¿Revisé la ortografía y presentación final?			

18. Recomendación final para los estudiantes

Este proyecto busca que los estudiantes apliquen los fundamentos de Programación Web I en un problema cercano a la realidad. Lo más importante no

es hacer una página complicada, sino una página clara, ordenada, interactiva y funcional.

El proyecto debe demostrar que el estudiante sabe estructurar una página con HTML5, diseñarla con CSS, hacerla responsiva, usar JavaScript para resolver un problema y manipular el DOM para mostrar resultados dinámicos al usuario.